

RWKV-7 State Semantic Embedding: A Unified Supervised Projection Framework for Three Tasks, with a Production Agentic Router

Zhenyuan Gu*

August 21, 2026

Abstract

RWKV-7, as a modern linear RNN architecture, contains two levels of internal states: (1) the *WKV state*—a recurrent memory matrix shaped $[\text{num_heads}, \text{key_dim}, \text{value_dim}]$ that is the core of the Delta Rule; and (2) the *hidden state*—the output vector of each TMix layer. This paper finds that the hidden state extracted by albatross (BlinkDL’s inference engine) contains rich semantic information but exhibits severe anisotropy, preventing unsupervised methods (KMeans, cosine similarity) from effectively mining it—achieving only $v\text{-measure} = 0.44$ on 20 Newsgroups (sklearn full-text version) clustering and Spearman = 0.46 on STS-B semantic similarity. As a comparison, we also tried various aggregation methods on the WKV state (Q-Readout, row_sum, diag, etc.), yielding even worse clustering performance (best 0.11), indicating that the hidden state is more suitable for semantic extraction than the WKV state. This paper presents a key insight: **hidden state contains semantic information (supervised MLP classification achieves 0.93), but unsupervised methods cannot extract it; supervised data is needed to train task-specific projectors to unlock its semantic potential.** Based on this insight, all three tasks achieve the following results: **(1) Semantic Similarity:** trained an STS-specific projector using 46.9k labeled pairs from NLI/STS/SICK-R (strictly deduplicated, removing 1,249 leak pairs overlapping with STS-B dev/test), achieving Spearman = 0.8188; **(2) Topic Clustering:** validated on two versions of 20 Newsgroups—MTEB short-text version (59,545 samples) supervised projection $v\text{-measure} = 0.9506$ (+217% over the unsupervised baseline 0.2912); sklearn full-text version (strict dedup + stratified 70/15/15 split) SupCon loss $v\text{-measure} = 0.6660$ (+50% over the unsupervised baseline 0.4434); **(3) Task Classification:** achieved test accuracy = 0.9325 on an independent test set using Hidden+MLP. It should be emphasized that STS and clustering tasks use supervised projectors and are thus supervised methods; the comparison with unsupervised baselines is only to validate the claim that hidden state requires supervised projection, not to claim superiority over unsupervised methods. Beyond offline benchmarks, the classification pipeline is deployed as the production smart router of an open-source desktop assistant: on a 2.9B int8 backbone with Chinese-language augmentation, held-out test accuracy reaches 0.960 (0.971 end-to-end through the deployed engine) at 77ms mean latency, with zero runtime dependencies beyond the already-resident local model. All offline results are based on the albatross inference engine (source code unmodified), a 0.4B RWKV-7 model, and CPU training (parameters $\sim 3.15\text{M}$), with all code open-source and reproducible.

*Corresponding author. Email: admin@ai00-x.com. Work done as an independent researcher; production deployment in the open-source ai00-x client (<https://github.com/cgisky1980/ai00-x-client>).

1 Introduction

1.1 Background

RWKV-7 [1] is a modern linear RNN that achieves $O(L)$ complexity sequence modeling through the Delta Rule [2]. It contains two levels of internal states:

- **WKV state** $S \in \mathbb{R}^{\text{num_heads} \times \text{key_dim} \times \text{value_dim}}$: recurrent memory matrix, the core of the Delta Rule, recursively updated at each time step.
- **Hidden state** $h \in \mathbb{R}^{\text{hidden_dim}}$: the output vector of each TMix (time mix) and CMix (channel mix) layer, an intermediate representation of forward propagation.

This paper studies the semantic extraction capability of the **hidden state**. The WKV state was also experimented with as a baseline (§4.3.3), but its clustering performance was much lower than the hidden state (0.11 vs 0.47). Albatross [3] is an efficient inference engine developed by BlinkDL, providing CUDA-accelerated WKV kernels and batch concurrent inference capability.

1.2 Problem

Although the albatross-extracted hidden state contains semantic information, it suffers from severe anisotropy:

Task	Unsupervised Method	Metric
Topic Clustering	Hidden + standardize + KMeans	v -measure = 0.47 (sklearn full-text)
Semantic Similarity	Hidden cosine	Spearman = 0.46
Task Classification	Hidden + MLP (supervised)	test acc = 0.93

Table 1: Anisotropy of the raw hidden state across tasks.

The supervised MLP classification achieving 0.93 proves that the hidden state does contain semantic information; however, unsupervised methods perform poorly on clustering and similarity tasks, indicating that **information requires nonlinear transformation to be extracted**.

1.3 Contributions

1. **Key Insight**: the anisotropy of the albatross hidden state can be mitigated through supervised projectors without modifying the inference engine.
2. **Task-Specific Projection**: STS learns similarity ranking (46.9k pairs, strictly deduplicated); clustering learns class separation (12.8k labeled samples, sklearn full-text version with strict deduplication + split)—the two objectives differ and cannot be mixed.
3. **Strict Experimental Paradigm**: STS training data is strictly deduplicated against STS-B dev/test (removing 1,249 overlapping pairs); clustering uses sklearn full-text 20NG with strict deduplication + stratified 70/15/15 split, where train is for training, dev for best_state selection, and test for held-out evaluation; classification adds an independent test set, with head selection using only dev.

4. **Three Tasks Results:** STS Spearman = 0.8188; clustering validated on two datasets (MTEB short-text $v = 0.9506$, sklearn full-text SupCon $v = 0.6660$, +217%/+50% over unsupervised baselines respectively); classification test acc = 0.9325. Note that STS and clustering use supervised projectors and are thus supervised methods.
5. **Systematic Unsupervised Comparison:** 7 categories and 30+ unsupervised methods (KMeans/PCA/UMAP/Whitening/DeepCluster) peak at only 0.33 on the MTEB short-text version, empirically confirming the limitation of unsupervised methods and supporting the necessity of supervised projection.
6. **Pure Python Implementation:** based on the albatross inference engine, bucketed by length for concurrency, 250 samples/s.
7. **Production Validation:** the classification head ships as the smart router of an open-source desktop assistant—on a 2.9B int8 backbone it reaches 0.960 test accuracy and 77 ms mean latency with zero extra runtime dependencies; two deployment findings (next-token logits of an untuned base model are unusable for tier classification; a language-distribution pitfall in training data causes a “Chinese $\rightarrow$ R0” shortcut) are documented for practitioners.

2 Related Work

2.1 RWKV and Albatross

RWKV [1] combines the Transformer’s parallel training with the RNN’s inference efficiency. RWKV-7 introduces the Delta Rule update:

$$S_t = \text{Diag}(w_t)S_{t-1} + \beta_t(v_t - S_{t-1}^\top k_t)k_t^\top. \quad (1)$$

Albatross [3] is the official PyTorch inference implementation, providing CUDA WKV kernels and `forward_seq_batch` for batch concurrent inference.

2.2 Embedding Evaluation Benchmarks

- **MTEB** [4]: Massive Text Embedding Benchmark.
- **STS-Benchmark** [5]: Semantic Textual Similarity, Spearman correlation.
- **20 Newsgroups** [6]: 20-category topic document clustering.
- **NLI / SICK-R** [7]: Natural Language Inference and sentence pair scoring.

2.3 Anisotropy and Contrastive Learning

- **Angle** [8]: angle-based contrastive loss, mitigating embedding anisotropy.
- **SimCSE** [9]: contrastive learning to alleviate anisotropy, but requires many negative samples.
- **Whitening** [10]: linear transformation to eliminate anisotropy, but destroys nonlinear structure.

2.4 Agentic Model Routing

SquillaRouter [15] routes each agent turn across four cost tiers (C0–C3) using 390 hand-crafted features with a LightGBM ranker and ONNX inference; its technical report shows 43–90% realized-cost reduction on agentic benchmarks while preserving task quality. Our production deployment (§5) adopts the same four-tier philosophy but replaces hand-crafted features and external runtimes with the native hidden state of the already-resident local model.

3 Method

3.1 Albatross Feature Extraction

Model: RWKV-7-g1 0.4B (hidden=1024, 24 layers, 16 heads, head_size=64).

Features:

- **Hidden state** $h \in \mathbb{R}^{1024}$: mean pooling of the last layer’s FFN output.
- **WKV state** $S \in \mathbb{R}^{16 \times 64 \times 64}$: recurrent memory matrix at layer 12.

Batch Concurrent Extraction: bucketed by token length, sequences within each bucket have identical length (no padding needed), using albatross’s `forward_seq_batch` for concurrent inference at 250 samples/s (vs 70 samples/s for a single sequence, $3.6\times$ speedup).

Important Note: this work entirely uses the albatross inference engine without modifying any source code. The feature extraction script only replicates the `forward_seq_batch` loop structure to simultaneously extract intermediate-layer state and hidden (the original function only returns logits).

3.2 Unified Projection Framework

Core idea: use a 2-hidden-layer MLP to project the hidden state into a 128-dimensional semantic space:

$$\text{emb} = \text{normalize}(\text{MLP}(\text{BatchNorm}(h))). \quad (2)$$

MLP Architecture (parameters $\sim 0.86\text{M}$):

```
Input(1024) -> BatchNorm -> Linear(1024->512) -> GELU -> LayerNorm
              -> Dropout(0.2) -> Linear(512->512) -> GELU -> LayerNorm
              -> Dropout(0.2) -> Linear(512->128) -> L2 Normalize
```

Training objectives differ by task (key design):

Task	Training Data	Loss	Temperature τ
STS	46.9k labeled pairs (scores 0–5, deduplicated)	Angle (regression)	0.50
Clustering	12.8k labeled samples (score 0/1, sklearn split)	Angle (contrastive)	0.30

Table 2: Task-specific training objectives.

Why not mix: STS learns “similarity ranking” (relative distance), while clustering requires “class separation” (absolute distance). Experiments show that transferring the STS projector to clustering fails (v -measure drops from 0.34 to 0.14, see §4.4).

3.3 Task 1: Semantic Similarity (STS)

Training Data (total 46,898 pairs after strict deduplication). Originally 48,147 pairs; after removing pairs overlapping with STS-B dev/test, 46,898 pairs remain (1,249 pairs removed, 2.59%):

Dataset	Original	After dedup	Score range	Source
STS-B train	5,749	5,076	0–5	GLUE
NLI train	10,000	9,952	1–5	SNLI/MultiNLI
extra_train	22,471	21,943	0–5	STS12–16
SICK-R	9,927	9,927	1–5	SICK-R

Table 3: STS training data composition.

Deduplication rule (strictest, single-sentence level): any training pair containing a sentence that appears in the STS-B dev/test sentence set is removed, preventing the model from memorizing evaluation sentence representations.

Angle Loss ($\tau = 0.50$, optimal for the albatross path, much higher than the Rust path’s 0.1):

$$\mathcal{L} = -\frac{1}{n} \sum_i \left[y_i \log \sigma \left(\frac{\cos \theta_i}{\tau} \right) + (1 - y_i) \log \left(1 - \sigma \left(\frac{\cos \theta_i}{\tau} \right) \right) \right], \quad (3)$$

where $\cos \theta_i = \text{emb}_1 \cdot \text{emb}_2$ (embeddings are L2-normalized), and y_i is the min-max normalization of the score: $y_i = (\text{score}_i - \min) / (\max - \min)$, unifying heterogeneous score ranges (0–5, 1–5) from different datasets into $[0, 1]$.

5-seed ensemble: train 5 models with different seeds (42, 123, 456, 789, 1024), average embeddings, and re-apply L2 normalization.

3.4 Task 2: Topic Clustering

Data Source: `sklearn.fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'))`, removing headers/footers/quotes to prevent the model from memorizing meta-information. Original 18,846 samples $\rightarrow$ filter 515 empty documents $\rightarrow$ text-normalized hash deduplication removes 78 (0.43%, cross-group cross-posts) $\rightarrow$ 18,253 samples.

Stratified split (70/15/15): train 12,777 / dev 2,738 / test 2,738. Dev is used for model selection (best_state), test for final evaluation (held-out, not involved in training).

Supervised contrastive learning: pair construction 50% same class (score=1), 50% different class (score=0); resample 80,000 pairs per epoch; Angle loss, $\tau = 0.30$; 5-seed ensemble.

Evaluation: on the held-out test set, use projection + KMeans to evaluate v -measure. *Note*: this method uses class labels to train the projector, making it supervised clustering—not directly comparable to unsupervised MTEB benchmarks (e.g., bge-large’s 0.57). The goal is to verify the semantic potential of the hidden state, not to propose a new unsupervised clustering method.

3.5 Task 3: Task Classification

Method: hidden state + MLP classifier (Top-K head selection + PCA is included as a comparison baseline).

- Hidden + MLP: $1024 \rightarrow 256 \rightarrow 4$, cross-entropy loss.
- Top-K head (comparison): evaluate each head’s accuracy, select the top-8 heads’ state (32,768 dims) $\rightarrow$ PCA to 256 dims $\rightarrow$ MLP.

Strict split (70/15/15): train 70% / dev 15% (head selection + early stopping) / test 15% (final evaluation, held-out). Head selection is ranked only on dev, never touching test, avoiding overfitting.

4 Experiments

4.1 Experimental Setup

- **Backbone:** RWKV-7-g1 0.4B (`rwkv7-g1d-0.4b-20260210-ctx8192.pth`, BlinkDL/rwkv7-g1).
- **Inference engine:** albatross (official PyTorch + CUDA WKV kernel).
- **Training hardware:** CPU (Intel i7-12700K) + RTX 2080 Ti (backbone inference only).
- **Training framework:** PyTorch 2.0+ (CPU mode for projection training).
- **State layer:** L12 (experimentally determined optimal).
- **Max sequence length:** 128 tokens (truncated).

4.2 Datasets

Task	Dataset	Samples	Usage
Clustering	20NG (sklearn full-text, dedup)	18,253	train/dev/test (70/15/15)
Clustering (unsup.)	20NG (MTEB short-text)	59,545	unsupervised evaluation
STS	STS-B train + NLI + extra + SICK-R	46,898 pairs	training
STS	STS-B dev/test	1,500 / 1,379	evaluation
Classification	golden_balanced	16,751	train/dev/test (70/15/15)

Table 4: Datasets used in the three tasks.

4.3 Main Results

4.3.1 STS Task

Method	Training Data	Test Spearman
Unsupervised hidden cosine	—	0.4600
MLP + STS-B train (5.7k, not deduplicated)	5,749	0.5818
MLP + all data (46.9k, strictly deduplicated, baseline)	46,898	0.8053
MLP + all data (optimal config)	46,898	0.8188

Table 5: STS results. Optimal config: hidden_dim=1024, output_dim=512, dropout=0.1, $\tau = 0.5$, 50 epochs, 5 seeds.

Conclusion: after strict deduplication (removing 1,249 pairs overlapping with STS-B dev/test), baseline Spearman dropped from 0.8504 to 0.8053 (−0.0451), and the optimal config dropped from 0.8680 to 0.8188 (−0.0492), confirming the impact of data leakage. The optimal config still

Seed	dev	test	ensemble
42	0.8125	0.7786	0.7786
123	0.8071	0.7791	0.8006
456	0.8021	0.7900	0.8127
789	0.8022	0.7808	0.8157
1024	0.8102	0.7874	0.8188

Table 6: STS per-seed results.

improves +0.0135 over the baseline, and the result is more honest, still significantly outperforming the unsupervised baseline (0.46).

Comparison with supervised embedding models:

Method	Type	STS-B Spearman	Parameters
Ours (RWKV-7 0.4B + projection, dedup)	supervised	0.8188	3.15M (projector)
bge-large-en-v1.5 [12]	supervised	~0.85	335M
all-MiniLM-L6-v2 [13]	supervised	~0.86	22M
Unsupervised hidden cosine	unsupervised	0.46	—

Table 7: Comparison with supervised embedding models. Supervised model data from the MTEB leaderboard [4]; training data and evaluation splits may differ slightly across models, so this is an approximate comparison.

Our method achieves STS-B Spearman of 0.8188 with only 3.15M projector parameters (the 0.4B language model is frozen; only the projector is trained), lower than bge-large-en-v1.5 (335M full fine-tuning) at ~0.85 and all-MiniLM-L6-v2 (22M) at ~0.86. The gap is mainly attributable to the representation capacity ceiling of the 0.4B language model and the training data scale.

4.3.2 Clustering Task (Supervised Projection + KMeans)

This paper evaluates clustering on two complementary datasets: the **MTEB short-text version** (59,545 samples, title-level; comparable with the MTEB leaderboard, data download is difficult) and the **sklearn full-text version** (18,253 samples, strict dedup + stratified 70/15/15 split; reproducible by anyone, held-out evaluation).

Dataset	Samples	Unsupervised baseline	Supervised (SupCon)
MTEB short-text	59,545	0.2912	0.9506 (AnglE)
sklearn full-text	18,253	0.4434 $\pm$ 0.0146	0.6660

Table 8: Clustering main results on two datasets.

Configuration: SupCon loss, $\tau = 0.07$, batch=320 (PK sampler, 20 classes $\times$ 16), dropout=0.1, 30 epochs, 5-seed ensemble.

Why the MTEB short-text v -measure = 0.9506 has no data leakage. Since this result is notably higher than the sklearn full-text version (0.6660), we detail the data split and evaluation pipeline here to confirm no leakage. A common confusion must be clarified first: an earlier version mistakenly labeled “Projection + standardize + KMeans = 0.8466” (a supervised-projection result)

Method	Training type	Test v -measure
Hidden + standardize + KMeans (baseline, 10 seeds)	unsupervised	0.4434 ± 0.0146
STS projection transfer (failed)	supervised (STS)	0.1424
Supervised contrastive (AngleE, $\tau=0.3$, 80k pairs)	supervised (clustering)	0.6373
Supervised contrastive (SupCon, $\tau=0.07$, batch=320)	supervised (clustering)	0.6660

Table 9: sklearn full-text detailed results.

Seed	dev v	test v	ensemble
42	0.6351	0.6402	0.6402
123	0.6374	0.6378	0.6528
456	0.6281	0.6458	0.6614
789	0.6338	0.6473	0.6647
1024	0.6355	0.6418	0.6660

Table 10: sklearn full-text per-seed results (SupCon).

as the unsupervised baseline. After rerunning the unsupervised baseline (10-seed KMeans) and the supervised projection (5 seeds), the **true unsupervised baseline of the MTEB short-text version is 0.2912**. The 0.8466 is a supervised-projection result, not an unsupervised baseline.

1. **Data split (64/16/20 stratified)**: the 59,545 samples are split via stratified sampling by class labels into train (38,109; only for projector training), dev (9,527; only for best_state model selection), and test (11,909; fully held-out, never used in training or model selection).
2. **Evaluation pipeline (unsupervised clustering + supervised metric)**: the projector is trained only on train; best_state is selected by v -measure on dev. Final evaluation runs on test: project test hidden states to 128-d via the trained projector, then cluster with **KMeans (unsupervised, $k=20$)**. **Class labels are used only to compute the v -measure metric, not in the KMeans clustering step**, so there is no label leakage.
3. **Why the high result is reasonable**: the MTEB short-text unsupervised baseline is only 0.2912, indicating severe hidden-state anisotropy. Supervised projection lifts v -measure from 0.2912 to 0.9506 (+217%), an even larger gain than the +50% on the sklearn full-text version. Short-text samples have fewer tokens and more concentrated semantics, making it easier for supervised projection to learn clear inter-class separation boundaries.
4. **No-overfitting evidence**: dev $v = 0.9497$ is close to test $v = 0.9506$ (after the 5-seed ensemble), indicating the best_state selected on dev performs equally well on test.

Analysis of differences between datasets: (1) the sklearn full-text supervised result (0.6660) is much lower than MTEB (0.9506), due to less training data (12k vs 38k) plus harder long-text clustering; (2) but the sklearn full-text unsupervised baseline (0.4434) is higher than MTEB (0.2912), because removing headers/footers/quotes yields cleaner samples; (3) both datasets validate the core thesis: supervised projection significantly outperforms unsupervised methods (sklearn +50%, MTEB +217%).

4.3.3 Unsupervised Clustering Methods Comparison

To validate the claim that unsupervised methods cannot extract clustering structure from the hidden state, we systematically compared 7 categories and 30+ unsupervised methods on the official MTEB 20NG short-text version (59,545 samples, title-level):

Method category	Best method	Best v -measure
Hidden + KMeans (baseline)	L23 + standardize	0.2912
Nonlinear feature transform	L23 + sign-sqrt + standardize	0.3181
Nonlinear dim. reduction (UMAP)	UMAP(32) + KMeans	0.3263
Multi-layer concatenation	Last3 (L16+L20+L23)	0.2553
Layer difference	L23-L0 + sign-sqrt	0.3087
Whitening / top- k PC removal	PCA whitening 256	0.1629
WKV state aggregation stats	row_mean + col_mean	0.1020
DeepCluster self-supervised iter.	Round 1 ensemble	0.3021

Table 11: Unsupervised methods comparison on the MTEB short-text version.

Analysis: (1) linear methods cap at ~ 0.32 , far below the unsupervised MTEB SOTA of 0.57; (2) nonlinear dimensionality reduction (UMAP 0.3263) offers no breakthrough, indicating the problem is not in the reduction method; (3) multi-layer concatenation (0.26) is worse than single-layer L23 (0.32)—shallow-layer noise dilutes deep-layer semantics; (4) DeepCluster pseudo-labels cannot self-improve (0.30 $\rightarrow$ 0.30); (5) WKV state aggregation statistics peak at 0.10—the WKV state’s value range is too small (std = 0.13); clustering information resides in the hidden state, not the WKV state.

Root cause: the 0.4B language model’s optimization objective is next-token prediction, not clustering. The semantic information in the hidden state requires nonlinear transformation (supervised projection MLP) to unlock. Unsupervised methods cap at 0.33 (MTEB short-text) / 0.44 (sklearn full-text) vs supervised projection 0.6660 (sklearn full-text, SupCon) / 0.9506 (MTEB, Angle).

4.3.4 Classification Task

Method	dev acc	test acc
Hidden + MLP	0.9381	0.9325
Top-8 head + PCA256 + MLP	0.9247	0.9208

Table 12: Classification results.

Analysis: hidden + MLP outperforms top- k head + PCA (0.9325 > 0.9208), indicating that the albatross hidden state already possesses good task separability—directly training a classifier on hidden is sufficient; top- k head selection + PCA dimensionality reduction actually loses some information. dev and test accuracies are close (0.9381 vs 0.9325), indicating no overfitting. This is consistent with the core insight in §6.1—the hidden state contains semantic information that can be extracted with a simple MLP (classification is a discriminative task that does not require projection to a semantic space).

4.4 Ablation Studies

4.4.1 STS Projection Transfer to Clustering (Failed)

This experiment uses a sampled subset of 2,000 samples (100 per class), different from the full test set (11,909 samples):

Method	<i>v</i> -measure
Baseline (hidden + standardize, sampled 2,000)	0.3426
STS projection + KMeans	0.1424
STS projection + standardize + KMeans	0.1305
STS projection + PCA(64) + KMeans	0.1284
Hidden + STS projection concat	0.2756

Table 13: STS projector transfer to clustering fails.

Failure reason: STS learns “similarity ranking” (relative distance); after L2 normalization, embedding std = 0.0884 is too small, compressing inter-class distances. Clustering requires “class separation” (absolute distance)—the two objectives differ.

4.4.2 Training Data Scale Impact (STS)

Training data	pairs	Test Spearman
STS-B train only	5,749	0.5818
+ NLI + extra + SICK-R (not deduplicated)	48,147	0.8504 (leaked)
+ NLI + extra + SICK-R (strictly deduplicated, baseline)	46,898	0.8053
+ NLI + extra + SICK-R (strictly deduplicated, optimal)	46,898	0.8188

Table 14: Training data scale impact on STS.

Conclusion: data volume is the key bottleneck for the STS task; an $8\times$ increase brings a +40% improvement (0.5818→0.8188). The non-deduplicated version 0.8504 was inflated due to training-data sentence overlap with dev/test; 0.8188 (optimal config) is the true generalization result.

4.5 Summary of Failed Directions

5 Production Deployment: Agentic Model Routing

The classification pipeline of §3.5 ships as the smart router of **ai00-x**, an open-source AI assistant desktop client with a resident local RWKV engine. This section reports the design, two deployment findings we believe are independently useful (§5.2, §5.3), and production measurements on a 2.9B backbone.

5.1 System Design

Each conversational turn is routed across four complexity tiers, following the agentic-routing philosophy of SquillaRouter [15]:

Method	Result	Failure reason
STS projection transfer to clustering	0.14	ranking vs separation
Unsupervised KMeans	0.29	severe anisotropy
Unsupervised hidden cosine (STS)	0.46	anisotropy
UMAP nonlinear dim. reduction	0.33	problem not in reduction
DeepCluster self-supervised iter.	0.30	pseudo-labels too weak
Multi-layer hidden concatenation	0.26	shallow noise dilutes
Whitening / top- k PC removal	0.16	degrades performance
Pure WKV state (Q-Readout)	0.11	state range small (std 0.13)
WKV state aggregation stats	0.10	no clustering info
Disable v_first mechanism	0.16	state std only 0.16
fp32 state (vs fp16)	0.13	precision not the root cause

Table 15: Summary of failed directions.

Tier	Semantics	Default target
R0	greetings, acknowledgments	local RWKV (zero API cost)
R1	simple single-step tasks	local RWKV
R2	code generation, multi-step reasoning	mid-tier model (user-configured)
R3	debugging, long-context, high-risk	primary model (user-configured)

Table 16: Four-tier routing semantics.

The pipeline: a zero-cost rule short-circuit intercepts trivial acknowledgments; otherwise the request is classified by RWKV; a post-processing rule stack applies a safety upgrade (predicted R0/R1 but $P(R2) + P(R3) > 0.45 \Rightarrow R2$) and a sticky-tier rule within a session; the tier maps to a model; any failure (engine not ready, head missing, dimension mismatch, timeout) falls back to the primary model.

The classifier runs **inside the already-resident inference engine**: tokenize (128-token truncation), prefill from a zero state on a dedicated slot isolated from the 16 generation slots, mean-pool the last-layer hidden state, and score it with the trained head. Unlike SquillaRouter [15]—390 hand-crafted features, a LightGBM/ONNX runtime, and a separate on-device embedding model—this design adds **zero runtime dependencies**: the head is a 0.66M-parameter matrix multiply (0.32 ms on CPU) over features of the model that must stay resident anyway for local generation.

5.2 Next-Token Logits Fail; State Embeddings Work

Before adopting state embeddings, we tested the seemingly simpler alternative: prompt the base model to answer with a tier label and read next-token logits. On the untuned 2.9B base model this fails categorically—a raw-prompt variant collapses to R0 (30% accuracy), a User:/Assistant:-wrapped variant collapses to R2 (28%). The next-token distribution of a small base model carries no trustworthy classification signal, while its hidden state yields 96%+ (§5.4). This mirrors the paper’s core claim at the output layer: the information is in the state, not on the surface.

5.3 A Language-Shortcut Pitfall

The golden_balanced training set contains 1,783 Chinese samples—all labeled R0. The first trained head duly learned a “Chinese $\rightarrow$ R0” shortcut: Chinese R1/R2/R3 requests collapsed to R0 with probability 1.00, while English traffic still scored $\sim 96\%$. Purely English evaluation cannot detect this failure. We generated 857 Chinese R1/R2/R3 template-composed augmentation samples and merged them into the training split only (held-out sets untouched). After retraining, a hand-crafted Chinese–English mixed set rose from 45% to 100%, and the English-dominant held-out set from 96.2% to 97.1%—no regression. We flag this as a general data-audit item for tier classifiers trained on mixed-language traffic.

5.4 Results at 2.9B

Scaling the frozen backbone from 0.4B (hidden 1024) to a 2.9B int8-quantized model (hidden 2560), same head architecture and training protocol (70/15/15 stratified split; AdamW 1e-3, cosine schedule, 30 epochs, early stopping on dev):

Backbone	dev acc	test acc
0.4B fp16 (§4.3.4)	0.9381	0.9325
2.9B int8 + zh augmentation	0.969	0.960

Table 17: Classification accuracy across backbones.

The comparison jointly changes backbone scale and augmentation; the unaugmented 2.9B head already classifies English traffic at $\sim 96\%$ but collapses on Chinese R1–R3 (§5.3), so most of the gain is attributable to the backbone, with augmentation repairing the Chinese failure mode.

End-to-end through the deployed engine: 97.1% on an independent 314-sample held-out set (per-class: R0 100% / R1 95% / R2 96% / R3 100%) and 100% on a 40-sample hand-crafted Chinese–English mixed set. The Python-trained weights and the Rust inference port agree exactly (max probability difference 0.000000).

Latency (RTX 2080 Ti; full pipeline tokenize $\rightarrow$ zero-state prefill $\rightarrow$ mean-pool $\rightarrow$ head; 250 samples after warm-up): mean 77 ms, p50 79 ms, p95 81 ms; throughput 12.4 classifications/s. Latency is nearly insensitive to request length (16 $\rightarrow$ 128 tokens: +12%) because the sequence-path GEMM pads the token dimension to 256. Stage breakdown at 128 tokens: prefill 93.8%, zero-state restore 5.8%, head 0.4%. Since R0/R1 turns are answered by the same local model, ~ 80 ms is negligible against local generation (seconds) or a cloud first token (500 ms+), and the 3 s routing timeout is never approached.

5.5 Deployment Notes

- **Graceful degradation:** a missing, corrupted, or dimension-mismatched head (e.g., after the user swaps the local model) silently disables classification and traffic falls back to the primary model; the system runs with or without the router.
- **Distribution:** the 14.2 MB head ships through the same content-addressed model manifest as the backbone (sha256-verified, mirrored to ModelScope/HuggingFace), adding ~ 14 MB to first-launch downloads.
- **Engine prerequisites:** two inference-engine bugs had to be fixed before the latency above was reachable—a kernel-cache invalidation that recompiled CUDA kernels (~ 0.5 s) on every

new sequence length, and a VRAM leak on variable-length prefills. Both affect any variable-length prefill workload, not only classification.

5.6 Comparison with SquillaRouter

	SquillaRouter [15]	Ours
Features	390 hand-crafted + external embeddings	native hidden state (2560-d)
Classifier	LightGBM + ONNX	0.66M-param MLP, pure Rust
Extra runtime deps	LightGBM, ONNX, embedding model	none
Headline result	cost -90% / -43% on agentic benchmarks	test acc 0.960, 97.1% end-to-end, 77 ms

Table 18: Comparison with SquillaRouter.

The two results are complementary rather than directly comparable: SquillaRouter reports cost-quality frontiers on agentic benchmarks; we report tier-classification accuracy and latency. The shared element is the four-tier philosophy; the engineering difference is that our router needs nothing beyond the local model the assistant already runs, with R0/R1 served at zero API cost. SquillaRouter’s self-improving data flywheel—online capture of routing outcomes and periodic router retraining—remains future work for our deployment.

6 Discussion

6.1 Core Insight: Supervised Projection Unlocks Hidden Semantic Potential

The anisotropy of the albatross hidden state (unsupervised STS only 0.46) is not a feature defect, but evidence that **unsupervised methods cannot extract nonlinear structure**. Supervised MLP classification achieving 0.94 proves the features contain information; only a supervised projector (MLP + Angle loss) is needed to map it to a linearly separable semantic space.

6.2 Task-Specific Projectors Cannot Be Mixed

Task	Objective	Projector behavior
STS	similarity ranking	preserves relative distances
Clustering	class separation	amplifies inter-class distances

Table 19: Task-specific projector behaviors.

STS projector transfer to clustering fails ($0.14 < 0.34$ baseline), proving that **projectors must align with task objectives**.

6.3 Data Scale Is Key

STS task expansion from 5.7k to 46.9k ($8\times$) brings a $+39\%$ improvement, indicating that the albatross hidden state’s semantic information requires sufficient data to extract through supervised learning. This is consistent with SimCSE’s finding that contrastive learning requires large amounts of samples [9].

6.4 Albatross vs Rust (web-rwkv)

The optimal $\tau = 0.50$ for the albatross path is much higher than the Rust path’s 0.1, rooted in hidden value-range differences (albatross std = 1.70 vs Rust std = 3.54). This work abandons comparison with Rust, using the albatross official implementation as the standard to avoid modifying the inference engine.

7 Conclusion

This paper proposes a supervised projection-based framework for RWKV-7 hidden state semantic embedding extraction, adopting a strict experimental paradigm (STS training data deduplication; clustering on the sklearn full-text version with strict deduplication + stratified split; classification with an independent test set), achieving the following results on three standard tasks:

- **Semantic similarity (supervised):** Spearman = 0.8188 (after deduplication), lower than bge-large-en-v1.5 (~ 0.85 , 335M) and all-MiniLM-L6-v2 (~ 0.86 , 22M), but with only 3.15M projector parameters. The gap is mainly attributable to the representation capacity ceiling of the 0.4B language model.
- **Topic clustering (supervised):** validated on two datasets—MTEB short-text v -measure = 0.9506 (+217% over the unsupervised baseline 0.2912); sklearn full-text SupCon loss v -measure = 0.6660 (+50% over the unsupervised baseline 0.4434). SupCon improves +0.0287 over Angle on the sklearn version.
- **Task classification:** test acc = 0.9325 (independent test set, head selection uses only dev).
- **Production routing (§5):** the same classification head, scaled to a 2.9B int8 backbone with Chinese augmentation and hardened by a rule stack, routes live traffic in a shipped desktop assistant—0.960 test accuracy, 97.1% end-to-end, 77 ms mean latency, zero extra runtime dependencies, R0/R1 served locally at zero API cost.

Core insights: the albatross hidden state contains semantic information but requires supervised projection to unlock; task-specific projectors cannot be mixed; data scale is the key bottleneck. The WKV state shows much lower clustering performance than the hidden state (0.11 vs 0.44), indicating the hidden state is more suitable for semantic extraction. All offline methods are based on a 0.4B model, the albatross inference engine (source code unmodified), CPU trainable with ~ 3.15 M parameters, suitable for edge deployment; the production router (§5) additionally validates the classification pipeline on a 2.9B int8 backbone in live traffic.

Honest disclosure. The early-version reported STS 0.8504 was inflated due to data leakage (STS training data overlapped with STS-B test by 1,249 pairs); the clustering 0.8466 was actually a “projection + standardize + KMeans” result that was mistakenly labeled as the unsupervised baseline. After rerunning the unsupervised baseline (10 seeds) and the supervised projection (5 seeds), and rerunning all three supervised clustering experiments for verification, the true unsupervised baseline of the MTEB short-text version is 0.2912. This version has been strictly deduplicated, uses held-out evaluation, and corrects the baseline-labeling error, giving more honest results.

References

- [1] RWKV. RWKV-7: Linear RNN with Delta Rule. <https://www.rwkv.cn/>.
- [2] Delta Rule. Wikipedia. https://en.wikipedia.org/wiki/Delta_rule.
- [3] BlinkDL. Albatross: Efficient RWKV-7 Inference Engine. <https://github.com/BlinkDL/Albatross>.
- [4] Muennighoff, N. et al. MTEB: Massive Text Embedding Benchmark. arXiv:2210.07316.
- [5] Cer, D. et al. SemEval-2017 Task 1: Semantic Textual Similarity Multilingual and Crosslingual Focused Evaluation. SemEval 2017.
- [6] 20 Newsgroups. <http://qwone.com/~jason/20Newsgroups/>.
- [7] Marelli, M. et al. SICK: A Sentiment Inference Corpus. LREC 2014.
- [8] Li, X. et al. AnglE-optimized Text Embeddings. arXiv:2309.12871.
- [9] Gao, T., Yao, X., and Chen, D. SimCSE: Simple Contrastive Learning of Sentence Embeddings. EMNLP 2021.
- [10] Su, J. et al. Whitening Sentence Representations for Better Semantics and Faster Retrieval. arXiv:2103.15316.
- [11] Reimers, N. and Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- [12] Xiao, S. et al. BAAI/bge-large-en-v1.5. <https://huggingface.co/BAAI/bge-large-en-v1.5>.
- [13] Wang, W. et al. sentence-transformers/all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.
- [14] MTEB Leaderboard. <https://huggingface.co/spaces/mteb/leaderboard>.
- [15] Liu, X. et al. Agentic Routing: The Harness-Native Data Flywheel. arXiv:2607.11399, 2026.

A Reproduction Guide

Requirements. Python 3.10+ with uv; a CUDA GPU (for albatross inference); on Windows, MSVC is required for CUDA extension compilation.

Reproduction commands.

```
# 1. Download model and albatross source
uv run --project ../../scripts python 00_setup.py

# 2. Extract features
./run_with_msvc.bat extract_features.py --task sts --sts-subdir
    sts_dedup --batch-size 16 --max-length 128
./run_with_msvc.bat extract_features.py --task cluster_sklearn
```

```

--batch-size 16 --max-length 128
./run_with_msvc.bat extract_features.py --task classification
--batch-size 16 --max-length 128 --limit 8000

# 3. Run three tasks
uv run python 02_sts_similarity.py --data-dir ../../data/sts_dedup
--device cuda --n-epochs 50 --hidden-dim 1024
--output-dim 512 --dropout 0.1          # STS: 0.8188
uv run python 07_cluster_supcon.py --device cuda    # Clustering: 0.6660
uv run python 03_classification.py                # Classification: 0.9325

```

Expected output.

```

STS:          5seed ensemble: Spearman = 0.8188
Clustering:   5seed ensemble: v_measure = 0.6660 (sklearn SupCon)
              / 0.9506 (MTEB Angle)
Classification: test_acc = 0.9325

```

Production router (§5). Feature extraction for the 2.9B backbone runs through the deployed Rust engine (`extract_router_features.rs` in the `rwkv-rsv` repository); training and Chinese augmentation live in the `ai00-x` client repository (`scripts/router_head/: train_mlp.py, gen_zh_augment.py`), using the same protocol as above. Production measurements (77 ms mean latency, 12.4 classifications/s) are from the deployed desktop client on an RTX 2080 Ti.